EFFICIENT IMPLEMENTATION FOR VIDEOCONFERENCING IN 3G WIRELESS NETWORKS

Weijia Jia, Haohuan Fu and Ji Shen

Department of Computer Science, City University of Hong Kong
83 Tat Chee Avenue, Kowloon, Hong Kong, SAR China
Email: itjia@cityu.edu.hk

Abstract*

This paper introduces the prototype of design and implementation of a videoconferencing (VC) system and protocol for 3G networks based on our 3G-324M protocol stack. Efficient multimedia processing and intelligent MCUs (multipoint control unit) for efficient token transfer among a group of terminals are presented. The prototype system has shown its performance for VC in WLAN environment.

1. Introduction

3G wireless multimedia communications are particularly referred to as the International Mobile Telecommunications 2000 (IMT-2000) that has been deployed and developed substantially [1]. ITU-T H.324 [2] is an umbrella protocol defined by International Telecommunications Union (ITU) to enable multimedia communication over low-bit rate terminals (in the following, we will drop "ITU-T" from the prefix of standard names for simplicity). H.324 and several mobile specific annexes are usually referred to as H.324M (M stands for *mobility*). The 3rd Generation Partnership Project (3GPP) is a body that comprises wireless infrastructure, handset and service providers throughout the world [13, 14]. 3GPP has adopted the H.324M with some modifications in codec and error handling requirements to create the 3G-324M standard for circuit-switched 3G networks. In order to support the enhanced and delay sensitive video services among the heterogeneous terminals, 3G video phones or terminals are required to support 3G-324M protocol stack. 3G-324M currently operates with WCDMA air interface, but can also operate on other 3G technologies because the 3G-324M call setup is able to reuse the underlined air interface protocol that the hand-held device uses. In addition to this, after establishing a call, several logical communication channels can be set up between the call participants. Through these channels, the call control information and multimedia streams can be reliably transmitted. Normally a 3G-324M enabled mobile phone/terminal consists of four parts:

- Bit-stream generator— is an embedded software/hardware that allows the connection between two different phones/terminals.
- Signaling channel— is used for the exchange of capabilities and opening of video, audio and data channels between two different phones. The signaling channel is defined by H.245 protocol.
- Data— is the audio and video codec and other data channels. These channels are actually what the phone users see or hear. They are also encapsulated in various standards (such as widely-known MPEG-4). Most of these solutions include software and hardware.
- Multiplexer—a software unit multiplexes and de-multiplexes the signaling and data channels together to send and receive them through air interface. The multiplexer is defined by H.223 protocol.

We have developed and implemented an efficient mobile multimedia transmission protocol stack based on 3G-324M standards using C++. In such implementation, the performance and quality of service are critical to the success of execution of the entire system. This paper discusses the efficient techniques and experiences of implementation for 3G-324M protocol stack. Our implementation has been tested in a realistic heterogeneous 3G communication environment in some Hong Kong and China industries for transmission of real-time video, audio and data and its performance is satisfactory.

The paper is structured into 5 sections. Sec. 2 introduces the 3G-324M protocol stack. Sec. 3 discusses the basic functions of implementing VC. Sec. 4 shows the efficient token transfer approach for the token transfer protocol and Sec. 5 concludes the paper.

2. What are 3G-324M and MCU?

The whole protocol stack of 3G-324M is shown in Fig. 1. ITU-T H.324 is a standard made by ITU-T for low bit rate multimedia communication, while H.245 and H.223 are two main parts under H.324 and have given specific descriptions about the procedures of message multiplexing

* .This effort is partially sponsored by City University of Hong Kong strategic grants 7001587, 7001709 and 7001777 and the National Basic Research Program (973) MOST of China under Grant No. 2003CB317003.

and transformation. However, H.324 is originally defined for multimedia communication for Public Switched Telephony Networks (PSTN), some of the specifications of this standard are not quite appropriate for the mobile terminals with low processing capability and power-constraints.

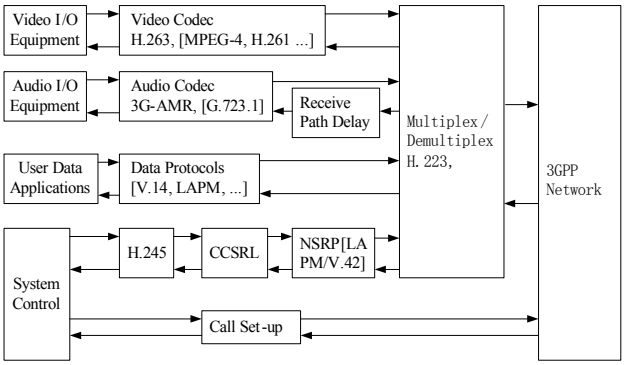

Fig. 1 3G-324M Protocol Stack

H.324 and its annex C are referred to as H.324M for mobile terminals. Thus H.324M is also an "umbrella standard" in respect with other standards which specify mandatory and optional video/audio codec, the messages to be used for call set-up, control and tear-down (H.245 [3]) and the way that audio, video, control and other data are multiplexed and demultiplexed (H.223 [4]). H.324M terminals offering video/audio communication will support H.263, H.261 video codec [8, 9]/G.731.1 audio codec [10] etc. In addition, other video and audio codec and data protocols can optionally be used via negotiation through exchange of the H.245 control messages. Note that the differences between 3G-324M and H.324M mainly lie in codec (voice by AMR- Adaptive Multi Rate Speech Codec, and video by H.263 or MPEG-4) and error handling requirements (H.223 Annex A and Annex B as mandatory) [14]. Therefore, 3G-324M inherits H.324M basically but must use AMR for speech codec. The AMR speech coder consists of the multi-rate speech coder, a source controlled rate scheme including a voice activity detector and a comfort noise generation system, and an error concealment mechanisms to combat the effects of transmission errors and lost packets [13].

2.1 Multipoint Control Unit

Multipoint Control Unit (MCU) supports multi-conferencing between three or more terminals and gateways. A two-terminal point-to-point conference can be expanded to a multipoint conference. The MCU consists of a mandatory Multipoint Control (MC) and optional Multipoint Processor (MP). The MC supports the negotiation of capabilities with all terminals in order to insure a common level of communications. It can control the resources in the multicast operation. The MC is not capable of mixing or switching of voice, video and data streams for a multipoint conference. There are three types of multipoint conferences controlled by MCU:

1) Centralized multipoint conference: All participating terminals communicate with the MCU point-to-point. The MC manages the conference, and the MP receives, processes, and sends the voice, video, or data streams to and from the participating terminals.

2) Decentralized multipoint conference: The MCU is not involved in this operation and the terminals communicate directly with each other through their own MCs. If necessary, the terminals assume the responsibility for summing the received audio streams and selecting the received video signals for display.

3) Mixed multipoint conference is a mix of the centralized and decentralized modes. The MCU keeps the operations transparent to the terminals.

2.2 Control of Multipoint Video Conferences through H.245

For multipoint conferences, all endpoints in the conference must exchange call signaling through MCU and the endpoint. Audio, video and data channels are open between the endpoints and the MP within the MCU. They are multicast to all endpoints in the conference. Initially, H.245 Control Channel is routed between the endpoints through gatekeepers. MCU-endpoint signaling is governed by H.245 control channel between endpoints and the MC within the MCU. When the conference switches to multipoint, the MC at the gatekeeper can be activated by the gatekeeper for subsequent operations. If one or both of endpoints have an MC, normal set up procedures is required. Fig. 2 shows that H.324 terminals may be used in multipoint configurations via interconnection through MCUs.

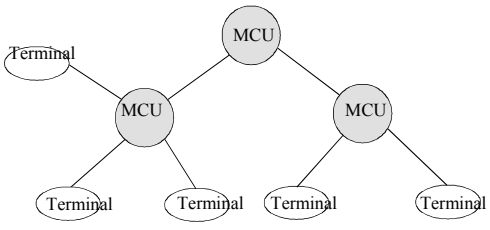

Fig. 2 Multipoint configuration

MCUs may force terminals into a particular common mode of transmission by sending to the terminal a receive capability set listing with the desired mode of transmission. 3G-324M terminals shall obey the

MultipointModeCommand message of H.245 as illustrated below. Since the modems on each link in a multipoint configuration may be operational at different bit rates, MCUs may choose to send H.245 *FlowControlCommand* messages to limit the transmitted bit rates to those which can be sent to receivers.

2.3 Multipoint Lip Synchronization

In a multipoint VC, each terminal may transmit different *H223SkewIndication* message for associated video and audio channels in H.223 protocol. To enable lip synchronization at receiving terminals, MCUs will transmit accurate *H223SkewIndication* messages. MCUs may accomplish this by adding delay to equalize the audio/video skew for all transmitting terminals. When switching between broadcasting terminals, H.223 may transmit a new *H223SkewIndication* message reflecting the audio/video skew of the current broadcaster.

3. Our System

3.1 Video/audio Processing based on PC Platform

Based on our previous implemented efficient transmission processing [5,6], in our prototype VC system, the implemented video and audio modules are demonstrated in Fig. 3:

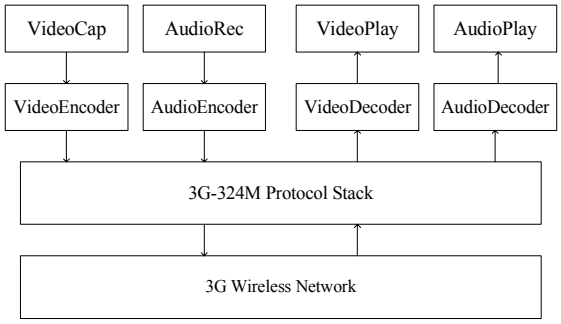

Fig. 3 Video/audio processing modules

Video/audio data can be captured, encoded and sent to the remote terminal through the 3G-324M protocol stack. Meanwhile, in the opposite direction, data received from the remote terminals will also be handled by 3G-324M protocol stack for decoding and displaying.

Video data is handled with Windows VFW APIs. And the waveform and auxiliary audio services of Windows APIs are utilized to handle the recording and playing of sound.

For audio codec, we applied basic waveform-audio Pulse Code Modulation (PCM) for testing the feasibility of our system. Therefore, more bandwidth may be consumed by audio. With a sampling frequency of 8 kHz, the audio bit rate is 64 kbps. We are currently adopting the 3G-324M AMR (Adaptive Multi-Rate) audio codec, which can greatly reduce the bandwidth taken by transmission of audio data. Currently, 3G communications only enable the point-to-point approach without having multipoint support for the applications such as the videoconferencing and group meeting. The term *multipoint communication* simply describes the interconnection of multiple terminals. Normally, a special network element, known as MCU, or simply a bridge, is required to provide this function.

3.2 Multi-thread Mechanism

From the general structure of VC as shown in Figure 3, we can see that the system has to handle several different tasks (such as video recording and displaying, audio recording and playback etc.) simultaneously. Thus, a multi-thread mechanism is used in the system. Four threads perform the following functions:

(1) Main thread: handling various messages and user interactions of Graphics Device Interface (GDI);

(2) Video capturing thread: capturing of a video frame and sending the frame to main thread with callback functions.

(3) Audio recording thread: recording of audio data. When an audio data buffer is full, the audio data is sent to the main thread with a callback function.

(4) Socket thread: listening to the data packets from remote terminal. If it detects a H.223 format packet, the data packet will be forwarded to H.223 module of 3G-324M protocol stack.

3.3. Token Transfer Protocol for VC

Consider a group of the participants (i.e. the members) join a VC and each time, only one member is allowed to make speech or to deliver its data information. To synchronize such group coordination, token transfer strategy is used for such coordination. We first give the model for the VC and then discuss some efficient token transfer approaches. The token transfer was implemented in H.245 control protocol using either logical control channel or data channel with piggy-backing approach.

3.3.1 VC and Token Transfer Model

Token is used for a member to hold in order to make a speech in the VC. A token is a logical entity that can be grabbed, released, passed, inhabited and queried. The group of member in a VC form a logical token ring such that

$$G= \{p1, \ldots, pn\}$$

where G is the VC group and p1,…, pn are the members. The members form a logical ring [p1, …, pn]. For each token pass along the ring, it is called a phase. We use P to denote the phase number for a conference with initialization 0. To pass the token on the logical ring, all members p1, …, pn maintain a bit array bitP = [b1,b2,…,bn] where bi is in {0,1}. In the array, bi=1 means that pi wishes or is assigned to hold the token to deliver its message or to make a speech in the current phase whereas bi=0 means that pi does not want to speak in VC in the current phase. In the following, we discuss two approaches for the token transfer: Centralized and decentralized transfers.

3.3.2 Centralized Token Transfer

In the centralized approach, a MCU plays a role of controller in the token transfer and is called PMCU. The PMCU is denoted as the member p0 as the initial token holder and it is known by all other members. Thus the bit array for token transfer is denoted by all members as bitP[PMCU] = [b0, …, bi, …, bn] and the token transfer can be piggybacked with the control messages.

Token passing strategy: We set the normal token transfer mechanism in the clockwise along the member logical ring [p0 = PMCU, p1, …, pn], for the token transfer or release order. In the centralized token passing, let *T* be the token position and the following operations are taken for the token transfer:

(1) *Token request*: Since the token is controlled by the PMCU, a member pi in VC, if it wishes to make a speech or deliver the data, will send a request bitP[i] to PMCU with bi=1. Upon reception of bitP[i], PMCU performs OR operations on the bit arrays and assigns to bitP[0] received bitP[i]s from all members. bitP[i] can be piggybacked with the data delivery of each member's message.

(2) *Token assignment*: PMCU multicasts the bitP[0] to the member group. The rest members, upon reception of bitP[0], know the token will be passed via the clockwise direction along the logical member ring p0, p1, …, pn provided bitP[j] =1 and the current token position *T=min{i where bi=1 in bitP[0]}*.

(3) *Token pass*: All members, upon reception of bitP[0] with bi=1, knows pi will hold the token and make speech after p(i-1) in phase P, i.e., the token position will jump to those pi for bi = 1 in these bitP[0] array.

(4) *Token return*: after the token passed the member pn, P increments and the token returns to PMCU.

(5) *Token pause and grab*: During the token pass, as long as there is any "0"s in the middle of the bit array, the token pauses. If PMCU wants to grab the token, it may issue a new bitP[0] with P increment. Otherwise, PMCU may keep salience and the token will transfer to the next member pj if bj = 1 on the logical ring.

We use the following examples to illustrate the situations:

Ex 1: bitP[PMCU]= [0…bi=1, 0…0, bj=1…0] means that both pi and pj wish to hold the token to make the speech. After pi has made the speech, the token is transferred to pj if PMCU does not grab the token during the pause period (i.e., j-i-1 "0" period).

Ex 2: bitP[PMCU]= [b0=1,…,bi=1, 0…0, bj=1…0] indicates PMCU will hold the token when pj has transmitted its information.

3.3.3 Decentralized token control

In the decentralized control, there is no role of PMCU and all peer members cooperate to deliver the data or to make the speech. Similar to *centralized approach*, a bit array is used for the control with the data delivered by each member.

(1) *Token initialization*: Each pi initializes P=0. Since there is no token controller, each member pi, no matter it wishes to send data or not, should multicast the bit array bitP[i]. Initially, p1, upon reception of bit arrays bitP[j] from rest pj where j=2,…, n, performs OR operations. Then p1 multicasts bitP[1] as the initial bit array to all members.

(2) *Token transfer*: Similar to centralized approach, as long as the initial bit array is multicast by all members, the token transfer follows the clockwise direction to rotate its position along the logical ring whenever the token passes back to p1, P increments.

(3) *Token piggyback*: A member pi, if it wishes to continue holding the token after its speech, it may piggyback the bit array on its data by setting b[i] = 1. Thus, other members pj, upon reception of the piggybacked bit array, will use OR operation to save the bit array in its record list bitP[j].

At a particular time instant, there is only one member that holds the token and multicasts messages or makes speech. The current token holder must decide to which member the token is to be transferred. In general, the current token holder must pass the token to a member who requests to hold the token for multicast messages/speech. In summary, the decentralized approach operates similar to centralized one and p1 carries partial job of PMCU expect that there is no token grab and all members have equal opportunity to hold the token.

3.4 Multiplexing and optimization in H.223

In our prototype VC system, H.223 protocol provides low delay and overhead by using segmentation, reassembly and combination of information from different logical channels

into a single packet. And H.223 also performs the multiplexing of multimedia data into bit-streams before transmission to air-interface. H.223 consists of Multiplex (MUX) Layer and Adaptation Layer (AL). AL is actually an interface for upper-layer applications and deals with different sources separately. The MUX layer performs the actual multiplexing. In this layer, data traffic from different sources can be multiplexed into one packet according to some rules which are exchanged by two terminals during the initialization of communication. These rules are described in the form of multiplex table. A multiplex table can contain at maximum 16 entries. The data structure of each entry is called a multiplex descriptor.

In H.223, a multiplex descriptor is in the form of an element list. Each element in the list either represents a slot of data from a specific information source, or is extended to be a sub-element-list. An element contains two data parts: {LCN#, RC#/RC UCF}. The first part LCN# (Logical Channel Number) specifies the information source; the second part RC#/RC UCF (RC means Repeat Count and UCF means Until Closing Flag) indicates the data slot length. For example, {LCN1, RC24} means 24 bytes from logical channel 1 that should be multiplexed into the packet and {LCN2, RC UCF} means that the bytes from logical channel 2 should be multiplexed into the packet until the end of the packet. The nested multiplex descriptor is easy to understand. However, when the descriptor structure is complicated, the processing overhead may be as high as the nested form is normally processed recursively.

To optimize the processing for the nested multiplex descriptor and avoid unnecessary recursive processing, in our implementation, a serialization approach is realized to re-structure a multiplex descriptor for efficient processing. Our approach is to identify the repeat pattern of an entry and make the right serialization processing. To identify the pattern, the key point is to understand that RC UCF only appears once in the multiplex descriptor. That is, exactly one part is repeated to the end of the packet. As a result, the whole multiplex descriptor can be divided into two parts: RC part, which is made up of elements having finite repeating count; and UCF part, which can be repeated until the end of the packet is reached. Consequently, the whole serialization process is divided into two steps. In the first step, the start point of UCF part must be identified and then, the whole descriptor is divided into RC and UCF parts. In the second step, the two parts are serialized into two separate lists of *Atoms*, one list for RC part, and another for UCF part. Here *Atom* is referred to as an indivisible element which can not be extended into a sub-list. The *Atom* structure contains three members: a *logical channel number* that specifies the information source; a *repeat count* which is a finite number that specifies how many bytes from that source will be filled in a packet; a *pointer* which links to the following atom. After the serialization process, the complicated nested table entry structures can be transformed into two straightforward linked lists of *Atoms*. The serialization process is shown in Fig. 4.

Comparison between original and serialized multiplex descriptor: The serialization of multiplex descriptor will introduce additional cost. This cost must be taken into consideration. In general, the serialization process may be invoked before the actual data communication; it may be counted as the initialization cost. With the original nested table entry structure, recursive function call is applied to the nested multiplex descriptors. Normally, mobile terminals have limited processing-power and our purpose is to reduce the recursive invocations. With serialization of multiplex descriptors, the operation of multiplexing is straightforward with less processing overhead.

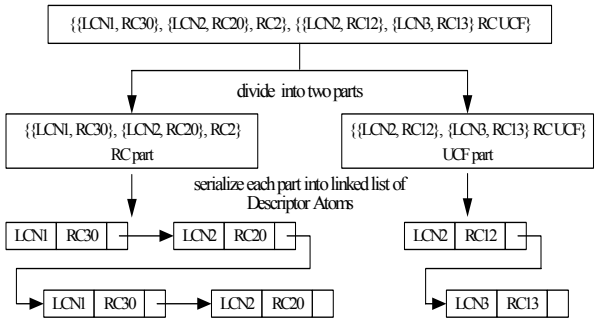

Fig. 4 Example of Serialization

As discussed above, there are two multiplex tables, one for sending and one for receiving data. Each table contains the maximum 16 different multiplex table entries, which define different multiplex patterns. Thus for each packet to be sent, one multiplex table entry has to be selected to do the multiplexing. In order to select the proper entry, each entry has a buffer associated. For the buffer arrangement, three factors have been taken into considerations:

(1) *Buffer size*: We wish to minimize the packets in the buffer as much as possible and the dynamic status of the buffer for each entry in H.223 entity must be closely monitored. If possible, we will select the most proper multiplex table entry to accommodate the current status.

(2) *Feasibility*: The multiplex entry should be feasible for the current situation. For example, entry 5 requires 50 bytes from logical channel LCN2 to multiplex the packets, but there are only 30 bytes from LCN2 in the buffer. In this case, entry 5 is not feasible.

(3) *Synchronization*: The multiplex pattern must synchronize audio and video data transmissions. With the same data in the video/audio buffers, different

multiplex entry may send video, audio data with different ratios. The multiplexing process should adjust the transmission rate of video/audio data to keep synchronization between the video/audio rates.

Currently, we apply IEEE 802.11 as the wireless media and the underlying network interface, and computers with microphone and camera as clients. Fig. 5 captures a live video conference and the log in the right text field records in H.245 protocol status.

Fig. 5 Videoconferencing Clients

As shown in Fig. 5, the system supports four members of video conferencing through 3G-324M. Table 1 shows the quality of the communication and contents.

Table 1. Video and Audio Parameters

Video Resolution	176*144*3
Video Codec	H.263
Video Bandwidth (Average)	40kbps
Audio Bandwidth (Average)	12bps

In the VC terminal, two major functions have implemented:

- Push-to-Talk for holding the token: This walkie-talkie style enables user to interfere with the VC manually. In general, attenuation is undesirable side effect when mixing the audios. With this function, we may get rid of audio noise among the different channels.
- Control Panel is open to play as a central conference controller. The functions in panel are included "Kick-off" and "Broadcast System Messages". The former is a function to expel any entity from the conference. The latter function is used when there is a text message to be forwarded among the group members.

5. Conclusion & Future Work

We have implemented a prototype of multiple terminal video conferencing based on IEEE 802.11 (WLAN) networks. Our implementation is feasible and we have applied optimization on the implementation for H.245 control and H.223 multiplexing protocols. We have applied both centralized and decentralized token transfer approaches for token transfer on top of H.245 when a member needs to make speech or deliver information. In the experiment, four terminals using H.223 are tested smoothly. The performance is satisfied with the initial implementations. Our current implementation has been tested in 3G networks in Hong Kong as well as some China industry.

6. References

[1] ITU-R Rec. PDNR WP8F, "Vision, Framework and Overall Objectives of the Future Development of IMT-2000 and Systems beyond IMT-2000", 2002.

[2] ITU-T Rec. H.324, "Terminal for low bit rate multimedia communication", March 2002.

[3] ITU-T Rec. H.245, "Control protocol for multimedia communication", July 2003.

[4] ITU-T Rec. H.223, "Multiplexing protocol for low bit rate mobile multimedia communication", July 2001.

[5] B. Han, H. Fu, J. Shen, P. O. Au and W. Jia, "Design and Implementation of 3G-324M - An Event-Driven Approach", IEEE VTC'04 Fall.

[6] W. Jia, H. Fu, B. Han, and P. Au, "Efficient Data Transmission Multiplexing in 3G Mobile Systems", Prod. Globe Mobile Congress, Oct. 2004, Shanghai, China.

[7] ITU-T Rec. T.120, "Data protocols for multimedia data conferencing", 1996.

[8] ITU-T Rec. H.263, "Video coding for low bit-rate communication", 1998.

[9] ITU-T Rec. H.261, "Video codec for audiovisual services at p × 64 kbit/s", 1993.

[10] ITU-T Rec. G.723.1, "Speech coders: Dual rate speech coder for multimedia communications transmitting at 5.3 and 6.3 kbit/s", 1996.

[11] ITU-T Rec. X.691, "Information technology – ASN.1 encoding rules – Specification of Packed Encoding Rules (PER)", 1997.

[12] ITU-T Rec. X.680, "Information Technology – Abstract Syntax Notation One (ASN.1) – Specification of basic notation", 1994.

[13] 3GPP TS 26.071 V4.0.0, "AMR Speech Codec; General Description", 2001.

[14] 3GPP TS 26.111 V5.1.0, "Codec for circuit switched multimedia telephony service: Modifications to H.324", June, 2003.

[15] Ly Q., Huang B., Wang F., "The mechanism of ASN.1 encoding & decoding implementation in network protocols", *Proceeding of International Conference on Information Technology: Coding and Computing*, pp 622-626, Apr. 2003.